LatticeBridge: Rare-Event Sequential Inference for Faithful Structured Sequence Synthesis*

Faruk Alpay
Department of Computer Engineering, Bahcesehir University
faruk.alpay@bahcesehir.edu.tr

Bugra Kilictas
Department of Computer Engineering, Bahcesehir University
bugra.kilictas@bahcesehir.edu.tr

April 2026

Abstract

Structured sequence generation often requires a model to satisfy several input-derived constraints in a single output. Standard decoding methods may assign high probability to fluent continuations while placing low mass on continuations that realize all required anchors jointly. We study this regime as a rare-event sequential inference problem. *LatticeBridge* combines a compact prefix language model, instance-compiled surface automata, and a twisted sequential Monte Carlo (SMC) decoder with resampling and multilevel splitting. The constraint representation remains input grounded and avoids manually curated topic lexicons or rule-based attribute classes. On 2,610 attainable validation tasks spanning CommonGen, E2E NLG, and WikiBio, the particle decoder improves exact anchor satisfaction and mean anchor coverage over greedy, beam-filtered, and best-of-k ancestral baselines under a shared proposal model. The gains occur with measurable changes in overlap-based quality, so the evaluation reports coverage, overlap, runtime, and particle diagnostics jointly. The resulting benchmark exposes constrained generation as an inference problem with an observable operating frontier rather than as a benchmark-specific decoding trick.

1. Introduction

Autoregressive models provide strong local continuation models, but structured generation often imposes conjunctive requirements that are not well represented by local likelihood alone. In data-to-text generation, for example, an output may need to realize several concepts or attribute values while remaining readable. A high-likelihood continuation can satisfy only a subset of those anchors, whereas a hard-constrained decoder can satisfy anchors at the cost of brittle search behavior. This paper treats the failure as a distributional issue: the set of fully satisfying continuations may have low probability under the base model even when the individual anchors are familiar.

We therefore formulate structured decoding as inference over a sequence space with a rare accepting event. Sequential Monte Carlo methods are well suited to this formulation because they maintain a weighted

*Code and benchmark files: https://github.com/farukalpay/latticebridge.

population of partial hypotheses and provide explicit diagnostics for weight degeneracy and resampling. Recent work has connected twisted SMC to inference in language models [Zhao et al., 2024, Wu et al., 2024]; related 2025–2026 work further studies learned twists and SMC-style decoding for constrained generation and model aggregation [Kim et al., 2025, Chan et al., 2026]. LatticeBridge focuses on a compact and reproducible setting: a small conditional proposal model, surface automata built from the input instance, and a particle decoder whose control signals remain inspectable.

The implementation separates data adaptation, constraint compilation, proposal modeling, and evaluation. This separation is important because benchmark-specific lexical rules can otherwise enter the decoding algorithm unnoticed. We use the following design constraints:

- constraints must be *instance driven*, extracted from the source example rather than from manually curated topical dictionaries,
- the library layer must stay reusable across tasks, while dataset adapters remain thin and explicit,
- fallback behavior must stay visible and measurable rather than silently degrading into ad hoc keyword logic.

LatticeBridge serializes each structured input into a prefix, trains a compact prefix language model, compiles anchor phrases into surface automata, and runs a twisted SMC bridge whose incremental reward depends on progress toward an accepting automaton state. The experiments evaluate coverage, fluency, runtime, effective sample size, and acceptance mass under a shared proposal model, so that the inference layer can be inspected independently of model scale.

2. Contributions

The paper makes five concrete contributions.

1. We formulate structured sequence synthesis as a sequential rare-event inference problem with a distance-to-acceptance potential defined by instance-compiled surface automata.
2. We introduce a compact prefix language model plus twisted SMC bridge that exposes resampling, effective sample size, and acceptance mass as first-class diagnostics.
3. We present a benchmark construction method for CommonGen, E2E NLG, and WikiBio that uses input-derived phrases only and keeps schema handling in thin dataset adapters.
4. We report multi-dataset validation results against greedy, beam-filtered, and best-of-k ancestral baselines, together with standard errors, runtime measurements, and particle-system diagnostics.
5. We release code and benchmark files at `https://github.com/farukalpay/latticebridge`, including the paper source, summary tables, generated figures, and a synthetic rare-event laboratory for controlled scaling studies.

3. Problem Formulation

Let x be a structured input, such as a set of concepts or attribute-value pairs, and let $y_{1:T}$ be a target surface sequence. A conditional language model $p_\theta(y_{1:T} \mid x)$ defines the base synthesis distribution. We are given a

set of input-grounded anchors

$$C(x) = \{c_1, \ldots, c_M\},$$

where each c_m is a surface phrase extracted from the source instance. The target objective is not simply to maximize model likelihood but to synthesize a sequence that is both plausible under p_θ and faithful to the anchors.

We encode satisfaction through an acceptance indicator

$$A(y_{1:T}; C) = \mathbf{1}\{\forall m,\ c_m \text{ appears in } y_{1:T}\}.$$

The exact conditional target would then be

$$\pi_T(y_{1:T} \mid x, C) \propto p_\theta(y_{1:T} \mid x)\, A(y_{1:T}; C),$$

but direct sampling from this distribution is typically intractable when $A = 1$ is rare.

To obtain a sequential bridge, we introduce an automaton state s_t after generating prefix $y_{1:t}$. The automaton tracks how much of each anchor phrase has been realized. Denote by $d(s_t) \geq 0$ the remaining distance to full acceptance. We then define a relaxed sequence target

$$\gamma_T(y_{1:T}) = p_\theta(y_{1:T} \mid x) \exp\{\lambda \Phi_T(y_{1:T})\}, \qquad \Phi_T(y_{1:T}) = -d(s_T),$$

where $\lambda > 0$ controls how strongly we bias toward exact acceptance. For $\lambda \to \infty$, the distribution concentrates on accepting trajectories when such trajectories exist.

The computational question is how to approximate this bridge under a finite particle budget while keeping the constraint representation task independent. We compile the surface constraints once, then run a twisted particle system that rewards progress toward acceptance at each generation step.

4. Constraint Compilation Without Hidden Heuristics

4.1. Instance-derived anchors

Anchor extraction is schema specific only in the sense that each dataset exposes a different structured input. CommonGen provides source concepts, E2E provides attribute values, and WikiBio provides titles plus infobox field values. After extraction, the inference layer receives only surface phrases and automaton states. No global topic lexicon, domain inventory, or manually curated attribute map enters the decoder.

For evaluation, we retain source phrases that appear in at least one reference surface. This attainability criterion prevents impossible exact-match labels in the validation split while leaving the constraint set instance specific. The decoder itself does not use the references.

4.2. Surface automata

Tokenization granularity can make phrase tracking unnecessarily brittle. A token-level automaton may fail when the tokenizer packs useful substrings into a single token. LatticeBridge therefore uses a surface automaton built over the emitted string fragments of tokenizer tokens. For each phrase c_m, we build a KMP-style automaton with states $0, \ldots, |c_m|$ that track the currently matched character prefix. The product state of all such automata defines the global constraint lattice.

This yields three operational advantages:

1. progress toward acceptance is visible through $d(s_t)$,
2. constraint logic stays reusable across datasets,
3. phrase tracking is no longer tied to a particular BPE segmentation accident.

5. Twisted Sequential Monte Carlo Bridge

5.1. Sequential target

Let y_t denote the token chosen at step t, and let the base model define one-step conditionals $p_\theta(y_t \mid y_{<t}, x)$. Define a progress signal

$$\Delta_t = d(s_{t-1}) - d(s_t),$$

which is positive when the new token moves the prefix closer to acceptance. The sequential potential used by the bridge is

$$G_t(y_t, s_{t-1}, s_t) = \exp\{\lambda \Delta_t\}.$$

The unnormalized path density becomes

$$\gamma_t(y_{1:t}) = p_\theta(y_{1:t} \mid x) \prod_{\tau=1}^{t} G_\tau.$$

5.2. Twisted proposal

Sampling directly from the base model and reweighting by G_t leads to rapid particle collapse. We instead use a twisted proposal

$$q_t(y_t \mid y_{<t}, x, s_{t-1}) \propto p_\theta(y_t \mid y_{<t}, x) \exp\{\kappa_t \Delta_t\},$$

where κ_t is the twisting coefficient. In the reported implementation,

$$\kappa_t = \tau,$$

with τ exposed as a benchmark parameter. The validation run sets $\tau = \lambda = 2.0$, so proposal shaping and the sequential target use the same local distance potential. Keeping τ explicit allows sensitivity experiments without changing the automaton or model code.

5.3. Importance weights and ESS

If particle i samples $y_t^{(i)} \sim q_t$, its incremental weight update is

$$\log w_t^{(i)} = \log w_{t-1}^{(i)} + \log p_\theta\left(y_t^{(i)} \mid y_{<t}^{(i)}, x\right) + \lambda \Delta_t^{(i)} - \log q_t\left(y_t^{(i)} \mid y_{<t}^{(i)}, x, s_{t-1}^{(i)}\right).$$

Normalized weights are

$$\bar{w}_t^{(i)} = \frac{w_t^{(i)}}{\sum_j w_t^{(j)}},$$

Algorithm 1 Twisted SMC bridge.

```
warm-start hidden states with source prefix
initialize P particles with automaton state s_0, hidden state h_0, and weight w_0 = 1
for t = 1 to T do
    for particle i = 1, ..., P do
        compute base logits from the prefix model
        compute progress scores Δ_t^(i) from automaton transitions
        sample y_t^(i) from twisted proposal q_t
        update particle weight using the importance ratio
        update automaton state and hidden state
    end for
    compute ESS and resample if needed
    apply elite splitting at scheduled checkpoints
end for
return best accepting sequence and summary diagnostics
```

and the effective sample size is

$$\mathrm{ESS}_t = \frac{1}{\sum_i (\bar{w}_t^{(i)})^2}.$$

We trigger resampling when $\mathrm{ESS}_t < \rho P$, where P is the particle count and ρ is the ESS threshold.

5.4. Multilevel splitting

In extremely rare settings, even twisted proposals can fail to maintain enough mass near acceptance. We therefore inject periodic multilevel splitting. At fixed intervals, particles are ranked by

$$R_t^{(i)} = \log w_t^{(i)} - \lambda d(s_t^{(i)}),$$

and a top-fraction elite set is replicated to refill the population. This rare-event device is related to adaptive multilevel splitting [Cerou and Guyader, 2007]. It changes the estimator bias profile, so we treat it as a finite-budget search component rather than as an exact posterior sampler.

6. Prefix Language Model

The proposal model consists of an embedding layer, a stacked GRU, and an output projection. This architecture provides a stable conditional proposal that can be trained quickly on local hardware and interrogated step by step during SMC. It also keeps hidden-state replication across particles inexpensive, which is important for local particle decoding.

Each example is serialized as

$$\langle \texttt{bos} \rangle \langle \texttt{src} \rangle \text{source serialization} \langle \texttt{tgt} \rangle \text{target surface} \langle \texttt{eos} \rangle.$$

The loss is a masked next-token cross-entropy in which the source prefix is excluded from supervision. The model therefore learns to predict the target continuation conditioned on the source serialization without having to model the serialization itself as natural text.

The recurrent proposal is used as a controlled conditional model for isolating the inference mechanism. It keeps hidden-state replication across particles inexpensive and avoids adding a second learned evaluator to the decoding loop.

7. Datasets and Benchmark Construction

7.1. Datasets

The reported structured-generation benchmark uses three public datasets:

- **CommonGen** [Lin et al., 2020]: concept-to-text generation,
- **E2E NLG** [Novikova et al., 2017, Dusek et al., 2019, Gehrmann et al., 2021]: restaurant meaning representations to text,
- **WikiBio** [Lebret et al., 2016]: biography infobox to text generation.

Dataset staging uses public dataset archives together with Hugging Face-hosted assets and tooling [Lhoest et al., 2021]. The quantitative claims in this paper use the three benchmark datasets listed in Table 1.

Table 1: Datasets used in the reported validation benchmark.

Dataset	Input form	Training records	Validation tasks
CommonGen	concept set	14,000	993
E2E NLG	attribute-value pairs	16,000	996
WikiBio	infobox record	20,000	621

7.2. Benchmark protocol

For each validation example, we select up to three anchor phrases from the source that also appear in at least one reference surface. This makes the exact acceptance event attainable while preserving an input-grounded constraint set. We then compare four inference methods:

1. greedy decoding,
2. beam filtering,
3. best-of-k ancestral sampling,
4. twisted SMC with resampling and splitting.

The reported metrics are:

- **success rate**: fraction of examples satisfying all anchors,
- **coverage**: mean fraction of anchors present in the chosen output,
- **ROUGE-L**: a simple overlap proxy for surface quality,
- **token-F_1**: additional overlap signal,
- **runtime**: mean wall-clock latency per example.

The validation run covers 2,610 attainable constraint tasks: 993 from CommonGen, 996 from E2E NLG, and 621 from WikiBio. Each summary reports the sample mean and standard error over examples.

8. Experimental Setup

All reported training, validation, and synthetic stress runs were executed on Apple Silicon using the Metal backend (`mps`). This hardware detail matters only operationally: it determines the local throughput of the particle system, not the definition of the bridge itself. The main model was trained for several epochs with a batch size of 48 and a maximum sequence length of 160 tokens. The validation benchmark used:

- beam size 6,
- best-of-16 ancestral sampling,
- 96 particles for twisted SMC,
- 64 maximum generated tokens,
- $\lambda = 2.0$ and twist scale $\tau = 2.0$,
- ESS resampling threshold $0.5P$, splitting interval 12, and elite fraction 0.2.

These settings define a fixed local-compute budget for comparing the four decoding methods. The complete run configuration is stored in `results/benchmark_structured_validation/validation_config.json`.

Table 2: Epoch-wise optimization trace for the compact prefix model.

Epoch	Train loss	Validation loss
1	3.993	4.035
2	2.980	3.756
3	2.673	3.592
4	2.455	3.478

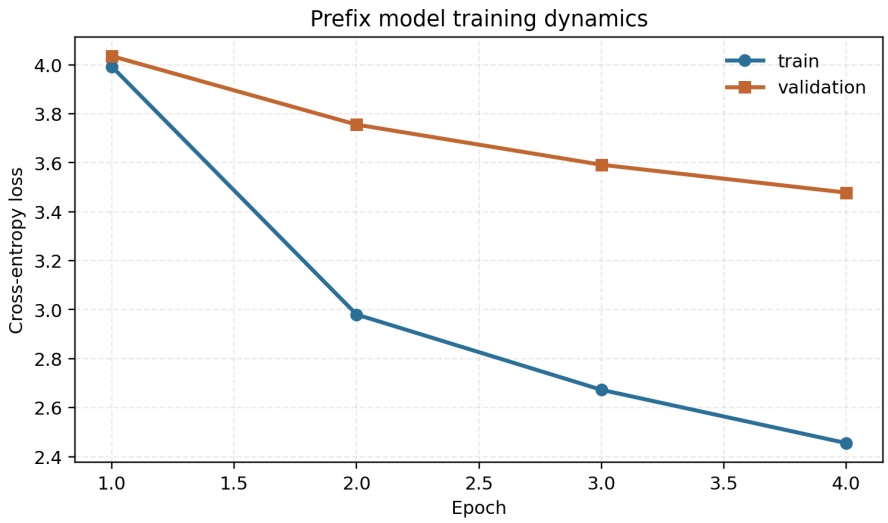

Figure 1: Training and validation loss for the compact prefix model.

9. Training Dynamics and Operating Frontier

Table 2 reports the optimization trace for the prefix model, and Figure 1 plots the same losses. The loss trace does not by itself certify convergence, but it is sufficient to rule out a grossly unstable proposal model.

The harder failure mode is combinatorial rather than merely lexical. The proposal model often emits individual anchors, but standard decoding under-explores continuations that realize several anchors jointly. The particle bridge is designed to probe that regime under a fixed proposal model.

Figure 2 gives a budget-oriented summary. On CommonGen, twisted SMC obtains substantially higher coverage and exact success than the tested baselines while remaining below the best-of-16 ancestral runtime. On E2E, twisted SMC improves exact success and coverage over greedy and beam filtering, and it obtains higher coverage than best-of-16 ancestral sampling at lower mean latency. We therefore report runtime together with coverage rather than treating satisfaction as an isolated metric.

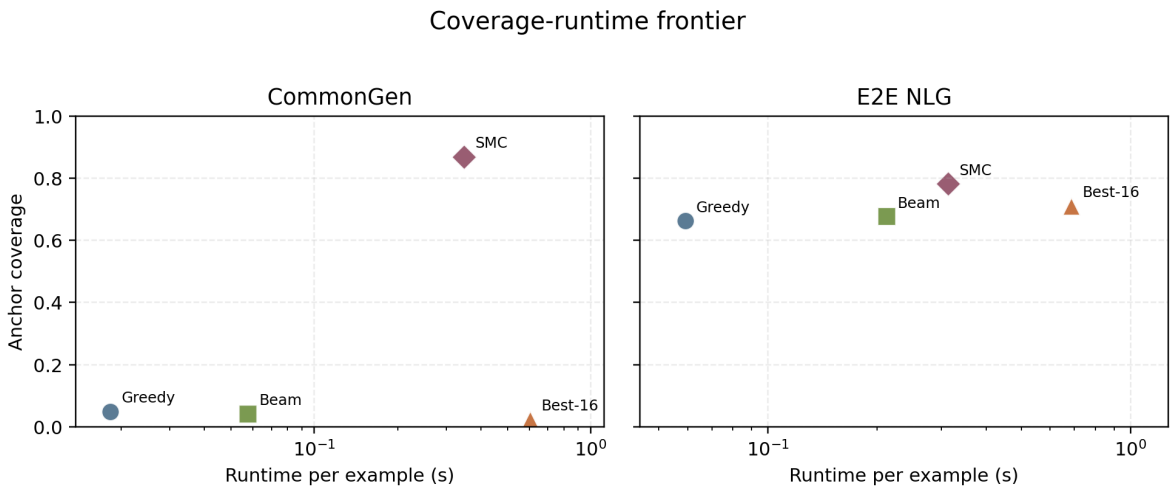

Figure 2: Coverage-runtime frontier for the validation benchmark. The x-axis is logarithmic to separate low-latency baselines from particle-based decoding regimes.

10. Results on Structured Generation

10.1. CommonGen

Table 3 shows the main validation result. On CommonGen, greedy, beam-filtered, and best-of-16 ancestral decoding do not produce any exact three-anchor successes in the 500-task validation subset. Twisted SMC raises exact success to 0.668 ± 0.021 and mean coverage to 0.869 ± 0.009. ROUGE-L is below greedy but comparable to beam filtering, indicating that the bridge reaches the constrained region without fully solving fluency under the compact proposal.

10.2. E2E NLG

On E2E, the base model already copies many structured attributes. Greedy decoding reaches 0.111 ± 0.014 exact success and 0.662 ± 0.007 coverage; beam filtering improves both slightly. Best-of-16 ancestral sampling reaches 0.407 ± 0.022 exact success. Twisted SMC obtains the highest exact success, 0.427 ± 0.022, and the highest coverage, 0.782 ± 0.009, with lower mean latency than best-of-16 ancestral sampling. Its ROUGE-L decreases relative to the copy-oriented baselines, so the table exposes the coverage–overlap trade-off directly.

Table 3: Validation benchmark over 996 attainable constraint tasks. Parenthesized values are standard errors. Bold marks the best success and coverage within each dataset.

Dataset	Method	Success	Coverage	ROUGE-L	Runtime (s)
CommonGen	Greedy	0.000 (0.000)	0.049 (0.005)	**0.252** (0.005)	0.018
	Beam filter	0.000 (0.000)	0.041 (0.005)	0.223 (0.004)	0.058
	Best-of-16 ancestral	0.000 (0.000)	0.023 (0.004)	0.179 (0.004)	0.605
	Twisted SMC	**0.668** (0.021)	**0.869** (0.009)	0.220 (0.004)	0.348
E2E NLG	Greedy	0.111 (0.014)	0.662 (0.007)	0.460 (0.007)	0.059
	Beam filter	0.139 (0.016)	0.676 (0.008)	**0.476** (0.007)	0.212
	Best-of-16 ancestral	0.407 (0.022)	0.709 (0.013)	0.381 (0.006)	0.686
	Twisted SMC	**0.427** (0.022)	**0.782** (0.009)	0.226 (0.004)	0.314

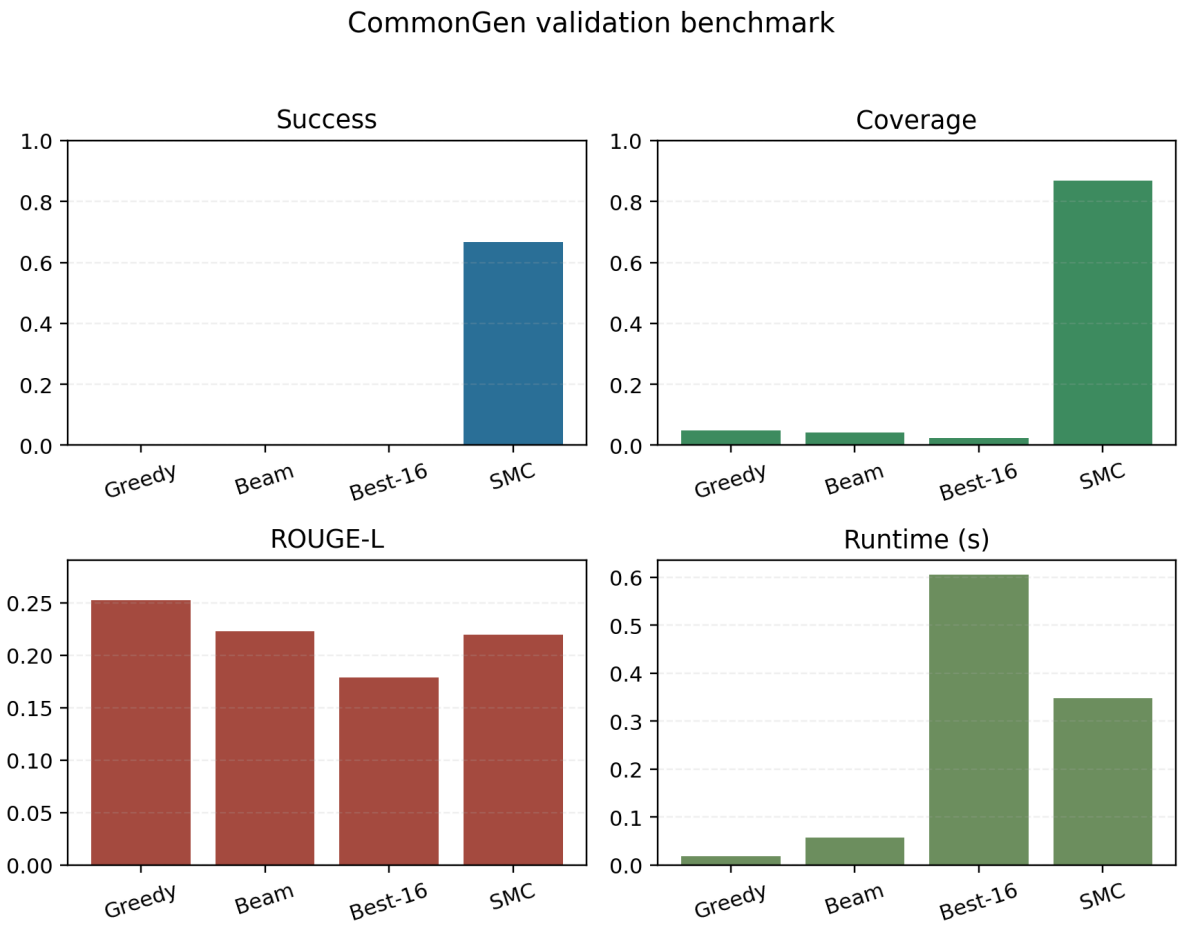

Figure 3: CommonGen validation benchmark by decoder.

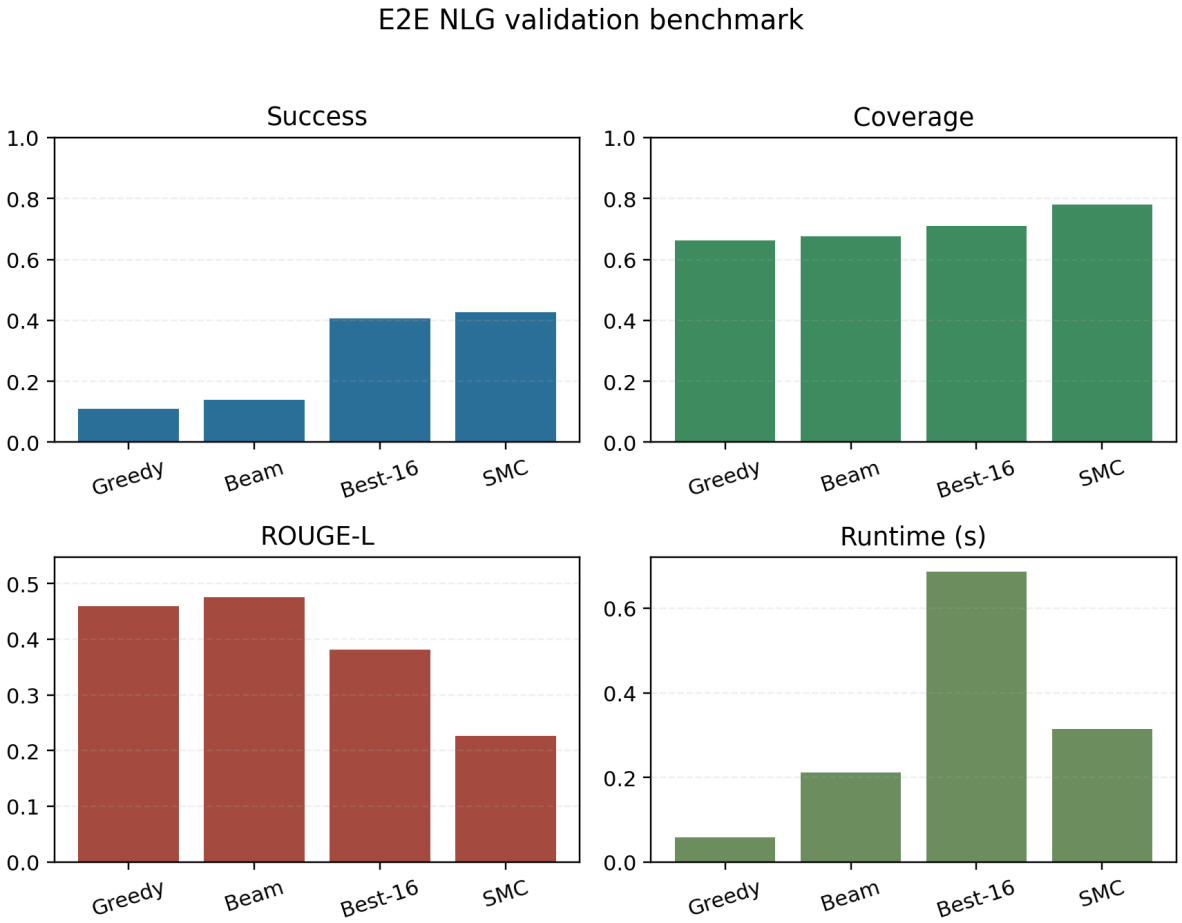

Figure 4: E2E NLG validation benchmark by decoder.

10.3. Interpretation

The validation results support three observations:

1. the proposal model can realize many anchors but does not reliably combine them under standard decoding,
2. SMC reweighting, resampling, and splitting increase exact satisfaction and mean coverage under the same checkpoint,
3. the bridge changes the operating point rather than dominating every metric: coverage rises, while ROUGE-L depends on how much the baseline already copies the structured input.

11. Synthetic Rare-Event Scaling

The implementation also includes a synthetic rare-event lab for controlled particle-system stress tests. The lab isolates acceptance-probability scaling, ESS behavior, and runtime under fixed tensor dimensions.

The three probe runs are summarized in Table 4. The exact acceptance estimate drops sharply with probe difficulty, while runtime rises and mean ESS remains in the thousands. The synthetic study is included to expose numerical scaling of the bridge under controlled barrier growth.

Table 4: Synthetic probe results from the compatibility launcher.

Probe	Exact accept	Relaxed accept	Mean ESS	Runtime (s)
Conflict 96/256/12/8192	0.0386	0.3419	5189.0	3.49
Scale 128/320/16/16384	0.0122	0.2410	10348.7	19.38
Scale 160/384/24/24576	0.000854	0.1690	15843.5	49.43

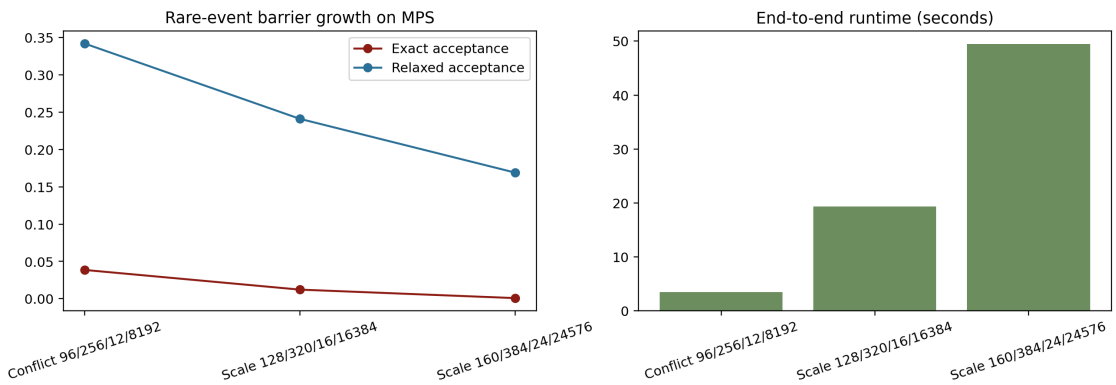

Figure 5: Rare-event barrier growth and runtime scaling for the synthetic probe family.

The probe results highlight two numerical properties:

- the exact–relaxed gap widens with probe difficulty, indicating that exact acceptance becomes progressively rarer,
- ESS remains in the thousands at the largest setting, so the particle system does not immediately collapse under this workload.

12. Qualitative Interpretation

The output examples in Appendix F illustrate the same control–quality trade-off seen in the aggregate metrics. Incomplete but fluent continuations often remain competitive under local likelihood, while the twisted bridge reallocates probability mass toward accepting sequences. The resulting outputs are therefore most informative when read together with coverage, overlap, acceptance mass, and ESS, rather than in isolation.

13. Connection to Doob Transforms and Dynamic Systems

Twisted proposals can be derived from the Doob h-transform associated with the probability of future acceptance. For the sequence model considered here, the exact harmonic function is not available because it depends on the probability of reaching an accepting automaton state from every possible recurrent hidden state and partial surface. LatticeBridge therefore uses distance reduction in the surface automaton as a computable surrogate.

From a dynamic-systems perspective, the partial sequence and automaton state define a controlled nonlinear state process

$$z_t = (h_t, s_t),$$

where h_t is the recurrent hidden state and s_t is the constraint state. The twist then acts as an online control law over the proposal kernel:

$$q_t(\cdot \mid z_t) = \mathcal{T}_\tau(p_\theta(\cdot \mid z_t), d(s_t)) .$$

This viewpoint determines which diagnostics are reported. Effective sample size measures degeneracy in the weighted particle approximation, acceptance mass estimates how much probability remains on accepting states, and runtime quantifies the hardware cost of applying the control law. Recent constrained decoding

work on accelerator-friendly automata and trie vectorization [Su et al., 2026] is relevant to this systems layer, while language-model SMC work [Zhao et al., 2024, Wu et al., 2024] provides the probabilistic inference background.

14. Approximate Doob Control Analysis

14.1. Exact Doob recursion

For a fixed horizon T, the variance-minimizing proposal for conditioning on acceptance is the Doob-transformed kernel induced by the future acceptance probability. Let

$$z_t = (h_t, s_t)$$

denote the recurrent hidden state and automaton state after step t. Define the harmonic bridge function

$$H_t(z_t) = \mathbb{P}_{p_\theta}(A(y_{1:T}; C) = 1 \mid z_t),$$

with terminal condition $H_T(z_T) = A(y_{1:T}; C)$. If $F(z_t, y)$ denotes the next state after emitting token y, then H_t obeys the backward recursion

$$H_t(z_t) = \sum_y p_\theta(y \mid z_t)\, H_{t+1}(F(z_t, y)).$$

The corresponding zero-variance proposal is

$$q_t^\star(y \mid z_t) = p_\theta(y \mid z_t) \frac{H_{t+1}(F(z_t, y))}{H_t(z_t)}.$$

Evaluating this proposal would require a backward recursion over the product of the language-model state and the automaton state. The automaton component is finite, but the recurrent hidden state is continuous, so the exact recursion is not tractable in the present implementation.

14.2. Distance-driven exponential twist

The implemented proposal replaces the unknown harmonic bridge with a surrogate defined by automaton distance. Writing the one-step progress as

$$\Delta_t(y) = d(s_t) - d(F_s(s_t, y)),$$

where F_s is the automaton transition, the implementation uses

$$q_t(y \mid z_t) \propto p_\theta(y \mid z_t) \exp\{\tau \Delta_t(y)\}.$$

Let

$$Z_t(z_t) = \sum_y p_\theta(y \mid z_t) \exp\{\tau \Delta_t(y)\}.$$

Because the sequence target itself uses the potential $\exp\{\lambda\Delta_t\}$, the incremental importance correction simplifies to

$$\log \frac{G_t(y)\, p_\theta(y \mid z_t)}{q_t(y \mid z_t)} = (\lambda - \tau)\Delta_t(y) + \log Z_t(z_t).$$

This identity separates two sources of variance:

1. the mismatch between the target bridge strength λ and the proposal twist strength τ,
2. the mismatch between local distance reduction and the true future acceptance value encoded in H_t.

In the validation benchmark $\tau = \lambda$, so the first term vanishes and the remaining correction is the proposal normalizer. The distance potential is not an estimate of the exact harmonic function; it is a deterministic control signal over the compiled automaton state.

14.3. Control-budget coupling

The coverage–latency trade-off can be formalized as a control problem with resource penalties. A useful surrogate objective is

$$\mathcal{J} = \mathbb{E}[\mathrm{cov}(y_{1:T})] - \eta_{\mathrm{lat}}\, \mathbb{E}[\mathrm{time}(y_{1:T})] - \eta_{\mathrm{deg}} \sum_{t=1}^{T} \mathbb{E}\left[\frac{P}{\mathrm{ESS}_t} - 1\right]_+,$$

where $\mathrm{cov}(y_{1:T})$ is anchor coverage, P is the particle count, and the last term penalizes degeneracy. The current implementation does not optimize this objective directly; it is used to interpret the reported coverage, latency, and ESS diagnostics under a common resource-sensitive criterion.

This view also distinguishes resampling from splitting. Resampling redistributes probability mass when normalized weights degenerate. Splitting reallocates finite compute toward high-scoring branches when acceptance becomes too rare under the current proposal. Both operations are therefore interpreted as control actions over a finite particle budget.

15. Related Work

15.1. Structured data-to-text generation

CommonGen [Lin et al., 2020], E2E NLG [Novikova et al., 2017, Dusek et al., 2019], and WikiBio [Lebret et al., 2016] are canonical benchmarks for testing whether a system can convert structured inputs into faithful text. Much work in this space focuses on architecture and pretraining rather than on the inference problem that appears after a model has already been trained.

15.2. Constrained decoding

Lexically constrained beam search [Hokamp and Liu, 2017, Post and Vilar, 2018], lookahead-based constrained decoding [Lu et al., 2022], search-oriented inference-time scaling such as A*-decoding [Chatziveroglou, 2025], and retrieval-constrained systems such as trie-based decoders [Su et al., 2026] all address subsets of the same issue: how to keep decoding inside a constrained region without destroying throughput. Our work differs in using a particle system that explicitly represents multiple partial trajectories and their weights.

15.3. Controlled language generation

Methods such as PPLM [Dathathri et al., 2020] and DExperts [Liu et al., 2021] alter decoding through gradients, auxiliary models, or expert/anti-expert logits. These methods are powerful but often depend on extra control models or attribute classifiers. LatticeBridge instead relies on instance-provided anchors and automaton geometry.

15.4. Sequential Monte Carlo and twisting

The theoretical backbone comes from Feynman–Kac particle systems [Del Moral, 2004], sequential Monte Carlo samplers [Del Moral et al., 2006], and modern tutorials such as Naesseth et al. [2019]. Twisted particle filters and related methods [Whiteley et al., 2016, Guarniero et al., 2017] motivate our use of future-oriented proposal shaping. Recent sequence-level extensions include twisted SMC for language-model inference [Zhao et al., 2024], reasoning-oriented twisted sampling [Wu et al., 2024], and 2026 work on language-model ensembling with shared character-space SMC [Chan et al., 2026]. Our implementation is closer in spirit to this language-model inference line than to classical state-space estimation, but the numerical concerns are the same.

16. Control Parameters and Validation Checks

The benchmark record stores the decoding controls used for every reported run. The validation configuration fixes the split, task count, maximum generation length, beam size, ancestral sample count, particle count, bridge strength, twist strength, ESS threshold, splitting interval, elite fraction, device, and random seed. These values are written to `results/benchmark_structured_validation/validation_config.json`; the aggregate task counts are written to `validation_tasks.json`.

The same checkpoint and tokenizer are used by all four decoders. Candidate selection is performed after decoding by the ordered key

$$(\mathbf{1}\{\text{all anchors realized}\},\ \log p,\ \text{ROUGE-L}),$$

so satisfaction is primary, model score is secondary, and reference overlap is used only to break remaining ties. This policy is applied uniformly to beam filtering, ancestral sampling, and SMC outputs.

For the SMC decoder, the validation run also records mean ESS, resampling count, acceptance mass, and realized generation length. On CommonGen, the mean ESS is 71.65 for 96 particles and the mean acceptance mass is 0.856. On E2E NLG, the mean ESS is 57.36 and the acceptance mass is 0.044, matching the lower exact-success margin in Table 3. These diagnostics make particle degeneracy visible rather than inferring it from coverage alone.

17. Conclusion

LatticeBridge treats faithful structured sequence synthesis as sequential rare-event inference rather than as a minor variant of ordinary decoding. The implementation combines a compact prefix model, surface automata, twisted SMC, and multilevel splitting in a form that keeps the control mechanism observable. In the multi-dataset validation benchmark, the bridge improves exact satisfaction and coverage relative to standard decoders while making the associated overlap and latency trade-offs explicit. The synthetic probe family

complements the benchmark by exposing how acceptance barriers, ESS, and runtime evolve under controlled scaling.

The main contribution is the controlled decomposition of the problem: input-derived constraints, a reusable automaton interface, explicit particle diagnostics, and benchmark files that report the quality–coverage–runtime trade-off under a fixed proposal model.

A. Implementation Checklist

- Manifest-driven dataset staging with external dotenv loading.
- Saved tokenizer and checkpoint files.
- JSON benchmark summaries and generated PNG figures.
- Compatibility launcher for the requested rare-event stress commands.
- Full manuscript source and bibliography in-repo.

B. Implementation Details

B.1. Tokenizer

We use a byte-level BPE tokenizer with special tokens for source and target delimiters. Byte-level modeling keeps the surface automata general enough to handle punctuation and mixed-case entity names without introducing language-specific preprocessing.

B.2. Model

The current checkpoint uses a 256-dimensional embedding, a 384-dimensional GRU hidden state, two recurrent layers, and dropout of 0.15. The objective is masked next-token prediction over the target portion only.

B.3. Inference

Greedy, beam-filtered, best-of-k, and twisted SMC all share the same prefix model. The only difference is how they explore the continuation space and how aggressively they bias toward acceptance. This makes the comparison cleaner than comparing distinct base models.

C. Evaluation Scope

The evaluation uses surface constraints rather than semantic classifiers, entailment models, or instruction-tuned judges. This choice narrows the claim: the paper evaluates exact realization of input-derived anchors, not semantic equivalence. The narrower setting is useful because it gives an unambiguous accepting event for rare-event inference and avoids introducing a second learned model as the evaluator.

Surface automata are weaker than semantic validators, but they are portable across datasets and make the acceptance state explicit. This is the appropriate abstraction for the present study because the decoder only needs an acceptance lattice and a transition function; dataset-specific schema handling remains outside the inference core.

Table 5: Budget-normalized lift relative to greedy decoding. Positive values indicate improvement beyond the greedy baseline.

Dataset	Method	Δ Success	Δ Coverage	Cov./extra s	Succ./extra s
CommonGen	Beam filter	0.000	-0.007	-0.19	0.00
	Best-of-16 ancestral	0.000	-0.025	-0.04	0.00
	Twisted SMC	0.668	0.820	2.48	2.02
E2E NLG	Beam filter	0.028	0.013	0.09	0.18
	Best-of-16 ancestral	0.296	0.047	0.08	0.47
	Twisted SMC	0.317	0.119	0.47	1.24

D. Budget-Normalized Diagnostics

Table 5 reports coverage and success lift relative to greedy decoding, normalized by additional runtime. On CommonGen, twisted SMC is the only tested method that reaches exact success. On E2E, best-of-16 ancestral sampling and twisted SMC both improve exact success, while twisted SMC gives the larger coverage lift at lower mean latency.

E. Example Diagnostics

CommonGen

Case	Anchors	Metrics	Baseline	LatticeBridge
exact lift	cat, couch, pet	cov 0.00->1.00; r-l 0.38->0.27; mass 1.00	A man is parked on a runway.	People dep on a couch that is located on the carpet
exact lift	building, climb, side	cov 0.00->1.00; r-l 0.24->0.15; mass 1.00	A room with a clock on the wall.	building b climb outside the city
exact lift	car, drive, snow	cov 0.00->1.00; r-l 0.31->0.43; mass 1.00	A man is a man in a field.	This snowboard is riding a car in the drive.
coverage lift	compete, crowd, front	cov 0.00->0.67; r-l 0.17->0.33; mass 1.00	a mountain arrivess	crowd genus in front of a compet eallowidming rev sherman with many items pounds .

E2E NLG

Case	Anchors	Metrics	Baseline	LatticeBridge
exact lift	Clowns, English, high	cov 0.33->1.00; r-l 0.27->0.29; mass 1.00	Bibimbap House serves English food in the riverside area near Clare Hall. It has a low ...	Clowns English food high customer rated worth bathroom near Clare Hall in the city ...
exact lift	Cocum, Chinese, moderate	cov 0.33->1.00; r-l 0.23->0.25; mass 1.00	Cocum is a coffee shop that serves Indian food in the high price range and is not kids ...	Cocum moderately Chinese food coffee shop, is family friendly and has a price range of ...
exact lift	Cotto, restaurant, Chinese	cov 0.33->1.00; r-l 0.37->0.14; mass 1.00	Cotto is a coffee shop that serves Indian food in the city centre near The Portland ...	Chinese restaurant Cotto near riverside and Indian food is The Cricketers. It has an ...
coverage lift	Alimentum, no, Burger King	cov 0.00->0.67; r-l 0.17->0.32; mass 0.00	The Green Man is a cheap, family friendly place with a price range of £20-25.	Alimentum has average ratings and is located near Burger King. Burger

The examples show the same trade-off measured in Table 3: the particle decoder moves probability mass toward accepting strings, while the compact proposal can still assign higher local probability to more fluent but incomplete strings. This is why the paper reports both exact satisfaction and overlap-based quality.

References

Frederic Cerou and Arnaud Guyader. Adaptive multilevel splitting for rare event analysis. *Stochastic Analysis and Applications*, 25(2):417–443, 2007.

Robin Shing Moon Chan, Tianyu Liu, Samuel Kiegeland, Clemente Pasti, Jacob Hoover Vigly, Timothy J. O'Donnell, Ryan Cotterell, and Tim Vieira. Ensembling language models with sequential monte carlo. *arXiv preprint arXiv:2603.05432*, 2026.

Giannis Chatziveroglou. A*-decoding: Token-efficient inference scaling. *arXiv preprint arXiv:2505.13672*, 2025.

Sumanth Dathathri, Andrea Madotto, Janice Lan, Jane Hung, Eric Frank, Piero Molino, Jason Yosinski, and Rosanne Liu. Plug and play language models: A simple approach to controlled text generation. In *Proceedings of ICLR*, 2020.

Pierre Del Moral. *Feynman-Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, 2004.

Pierre Del Moral, Arnaud Doucet, and Ajay Jasra. Sequential monte carlo samplers. *Journal of the Royal Statistical Society: Series B*, 68(3):411–436, 2006.

Ondrej Dusek, David M. Howcroft, and Verena Rieser. Semantic noise matters for neural natural language generation. In *Proceedings of the 12th International Conference on Natural Language Generation*, pages 421–426, 2019.

Sebastian Gehrmann, Tosin Adewumi, Karmanya Aggarwal, Pawan Sasanka Ammanamanchi, Aremu Anuoluwapo, Antoine Bosselut, Khyathi Raghavi Chandu, Miruna Clinciu, Dipanjan Das, Kaustubh Dhole, et al. The gem benchmark: Natural language generation, its evaluation and metrics. In *Proceedings of the 1st Workshop on Natural Language Generation, Evaluation, and Metrics*, pages 96–120, 2021.

Peter Guarniero, Adam M. Johansen, and Anthony Lee. The iterated auxiliary particle filter. *Journal of the American Statistical Association*, 112(520):1636–1647, 2017.

Chris Hokamp and Qun Liu. Lexically constrained decoding for sequence generation using grid beam search. In *Proceedings of ACL*, pages 1535–1546, 2017.

Sooyeon Kim, Giung Nam, Byoungwoo Park, and Juho Lee. Improving constrained language generation via self-distilled twisted sequential monte carlo. *arXiv preprint arXiv:2507.02315*, 2025.

Remi Lebret, David Grangier, and Michael Auli. Generating text from structured data with application to the biography domain. *arXiv preprint arXiv:1603.07771*, 2016.

Quentin Lhoest, Albert Villanova del Moral, Yacine Jernite, Abhishek Thakur, Patrick von Platen, Suraj Patil, Julien Chaumond, Mariama Drame, Julien Plu, Lewis Tunstall, et al. Datasets: A community library for natural language processing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 175–184, 2021.

Bill Yuchen Lin, Wangchunshu Zhou, Ming Shen, Pei Zhou, Chandra Bhagavatula, Yejin Choi, and Xiang Ren. Commongen: A constrained text generation challenge for generative commonsense reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 1823–1840, 2020.

Alisa Liu, Maarten Sap, Ximing Lu, Swabha Swayamdipta, Chandra Bhagavatula, Noah A. Smith, and Yejin Choi. Dexperts: Decoding-time controlled text generation with experts and anti-experts. In *Proceedings of ACL-IJCNLP*, pages 6691–6706, 2021.

Ximing Lu, Peter West, Rowan Zellers, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Neurologic a*esque decoding: Constrained text generation with lookahead heuristics. In *Proceedings of NAACL-HLT*, pages 780–799, 2022.

Christian A. Naesseth, Fredrik Lindsten, and Thomas B. Schon. Elements of sequential monte carlo. *Foundations and Trends in Machine Learning*, 12(3):307–392, 2019.

Jekaterina Novikova, Ondrej Dusek, and Verena Rieser. The e2e dataset: New challenges for end-to-end generation. In *Proceedings of SIGDIAL*, pages 201–206, 2017.

Matt Post and David Vilar. Fast lexically constrained decoding with dynamic beam allocation for neural machine translation. In *Proceedings of NAACL-HLT*, pages 1314–1324, 2018.

Zhengyang Su, Isay Katsman, Yueqi Wang, Ruining He, Lukasz Heldt, Raghunandan Keshavan, Shao-Chuan Wang, Xinyang Yi, Mingyan Gao, Onkar Dalal, Lichan Hong, Ed Chi, and Ningren Han. Vectorizing the trie: Efficient constrained decoding for llm-based generative retrieval on accelerators. *arXiv preprint arXiv:2602.22647*, 2026.

Nick Whiteley, Anthony Lee, and Karolina Heine. Twisted particle filters. *The Annals of Statistics*, 44(2): 822–859, 2016.

Xuansheng Wu, Wenlin Yao, Jianshu Chen, Xiaoman Pan, Xiaoyang Wang, Ninghao Liu, and Dong Yu. Step-by-step reasoning for math problems via twisted sequential monte carlo. *arXiv preprint arXiv:2410.01920*, 2024.

Stephen Zhao, Rob Brekelmans, Alireza Makhzani, and Roger B. Grosse. Probabilistic inference in language models via twisted sequential monte carlo. In *Proceedings of the 41st International Conference on Machine Learning*, pages 60704–60748, 2024.